

Assignment 1 – Practical Questions

After 8 Classes

Question 1: Employee Management System

Design a class `Employee` with the following specifications:

Private data members:

- `name` (string)
- `salary` (double)
- `empId` (int)

Static data member:

- `totalEmployees` - tracks how many `Employee` objects currently exist.

Requirements:

1. Write a **parameterized constructor** that initializes `name` and `salary`, auto-generates `empId` (e.g., using `totalEmployees + 1`), and increments `totalEmployees`.
2. Write a **copy constructor** (use `const` correctly) that properly duplicates an employee's data.
3. Write a **destructor** that prints a message ("Employee [name] removed") and decrements `totalEmployees`.
4. Write a setter `setSalary(double s)` that only updates salary if `s > 0`; otherwise print an error. Use the `this` keyword explicitly inside it.
5. Write a getter `getSalary() const` - must be a `const` function.
6. Write a **static function** `getTotalEmployees()` that returns the current count (callable without an object).
7. Write a member function `giveRaise(Employee &other, double amount)` that increases **another employee's** salary directly by accessing its private `salary` member (demonstrate that private access is class-based, not object-based).
8. In `main()`:
 - Create 3 `Employee` objects.
 - Print total employees using the static function.
 - Use `giveRaise()` to give one employee a raise from within another employee's function call.

- Put one employee inside a nested block { } so its destructor fires early - print total employees before and after to confirm the count updates.

Question 2: Rectangle Comparison and Method Chaining

Design a class `Rectangle` with the following specifications:

Private data members:

- `length (double)`
- `width (double)`

Requirements:

1. Write a **default constructor** and a **parameterized constructor**.
2. Write setters `setLength(double l)` and `setWidth(double w)` that validate the value is positive (else print an error), using `this->` inside each.
3. Write getters `getLength() const` and `getWidth() const`.
4. Write a function `double area() const` that returns `length * width`.
5. Write a function `compareArea(const Rectangle &other) const` that compares this rectangle's area to another's (passed by **const reference**) and prints which is larger - this function must **not** modify either object.
6. Implement **method chaining**: make `setLength()` and `setWidth()` return `Rectangle&` (reference to `*this`), so a user can write:
`Rectangle r;r.setLength(10).setWidth(5).display();`
8. Write a function `demonstratePassByValue(Rectangle r)` and a function `demonstratePassByReference(Rectangle &r)` - inside each, modify `r`'s length to 999. Call both from `main()` and print the **original object's** length after each call to show the difference in behavior.
9. In `main()`:
 - Create two rectangles using chained setter calls.
 - Compare their areas using `compareArea()`.
 - Call both pass-by-value and pass-by-reference functions and observe/print the difference.